

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Метрология, стандартизация и сертификация в
информационных технологиях»

ОТЧЕТ

по лабораторной работе №1
«Метрики размера программ»

Подготовили:

Видилин А.В.,
Варава А.В.

Проверила:

Болтак С.В.

Минск 2026

Цель работы: изучить метрики размера программ на примере метрики Холстеда.

Задание:

Разработать программу на языке программирования **TypeScript**, содержащую все основные операторы языка, а также пользовательские подпрограммы.

Парсер может быть реализован на любом языке программирования и должен иметь графический интерфейс. Для анализа взять код на заданном языке.

На основе анализа программа должна рассчитать метрики Холстеда. 6 базовых и 3 расширенных. Результат работы программы - таблица, как в примере, в которой должны отображаться 6 основных метрик Холстеда. Отдельно выводятся 3 расширенные метрики.

Код анализируемой программы на языке **TypeScript**:

```
interface DataMetrics {  
    mean: number;  
    variance: number;  
    min: number;  
    max: number;  
}
```

```
function calculateMean(data: number[]): number {  
    let totalSum: number = 0;  
    let count: number = 0;  
    for (let i: number = 0; i < data.length; i = i + 1) {  
        totalSum = totalSum + data[i];  
        count = count + 1;  
    }  
    if (count === 0) {  
        return 0;  
    }  
    return totalSum / count;  
}
```

```
function calculateVariance(data: number[], meanValue: number): number {  
    let sumSquaredDiff: number = 0;  
    let count: number = 0;  
    for (let i: number = 0; i < data.length; i = i + 1) {  
        let diff: number = data[i] - meanValue;  
        sumSquaredDiff = sumSquaredDiff + (diff * diff);  
        count = count + 1;  
    }  
    if (count === 0) {  
        return 0;  
    }
```

```
    }  
    return sumSquaredDiff / count;  
}
```

```
function findMinMax(data: number[]): [number, number] {  
    if (data.length === 0) {  
        return [0, 0];  
    }  
    let minVal: number = data[0];  
    let maxVal: number = data[0];  
    for (let i: number = 1; i < data.length; i = i + 1) {  
        if (data[i] < minVal) {  
            minVal = data[i];  
        }  
        if (data[i] > maxVal) {  
            maxVal = data[i];  
        }  
    }  
    return [minVal, maxVal];  
}
```

```
function filterPositiveNumbers(data: number[]): number[] {  
    let result: number[] = [];  
    let index: number = 0;  
    while (index < data.length) {  
        let current: number = data[index];  
        if (current > 0) {  
            result.push(current);  
        }  
        index = index + 1;  
    }  
    return result;  
}
```

```
function processDataset(numbers: number[]): void {  
    console.log("--- Начало обработки данных ---");  
  
    let filtered: number[] = filterPositiveNumbers(numbers);  
    let dataLength: number = filtered.length;  
    console.log("Количество положительных элементов:", dataLength);  
  
    if (dataLength === 0) {  
        console.log("Массив не содержит положительных чисел!");  
        return;  
    }  
}
```

```

    }

    let meanVal: number = calculateMean(filtered);
    let varianceVal: number = calculateVariance(filtered, meanVal);
    let minMax: [number, number] = findMinMax(filtered);

    let score: number = (minMax[1] - minMax[0]) * meanVal / (varianceVal + 1);
    console.log("Комплексный показатель (Score):", score);

    if (score > 50) {
        console.log("Оценка: Высокий показатель разброса");
    } else if (score > 20) {
        console.log("Оценка: Средний показатель разброса");
    } else {
        console.log("Оценка: Низкий показатель разброса");
    }

    console.log("--- Обработка завершена ---");
}

function main(): void {
    let rawData: number[] = [-10, 15, 3, -5, 42, 18, -2, 7, 0, 25, 30, -12, 8];
    processDataset(rawData);
}

main();

```

Код логики парсера:

Код логики парсера вынесен в отдельный файл и лежит рядом с отчётом в репозитории.

Расчет базовых метрик Холстеда, проведенный вручную, для данной программы приведены в таблице 1.

Таблица 1 – Расчет базовых метрик Холстеда

j	Оператор (f1j)	f1j	i	Операнд (f2i)	f2i
1	calculateMean()	2	1	string_literal	8
2	for()	3	2	array_literal	6
3	if()	9	3	data	18
4	calculateVariance()	2	4	totalSum	4
5	findMinMax()	2	5	0	14
6	filterPositiveNumbers()	2	6	count	10
7	while()	1	7	i	18
8	result.push()	1	8	length	6
9	processDataset()	2	9	1	8
10	console.log()	8	10	meanValue	2
11	main()	2	11	sumSquaredDiff	4
12	[]	9	12	diff	3
13	=	30	13	minVal	4
14	<	5	14	maxVal	4
15	.	6	15	result	2
16	+	9	16	index	5
17	===	4	17	current	3
18	/	3	18	numbers	2
19	-	2	19	filtered	5
20	*	2	20	dataLength	3
21	>	4	21	meanVal	3
22	function	6	22	varianceVal	2
23	let	20	23	minMax	3
24	return	8	24	score	4
25	else	2	25	50	1
26			26	20	1
27			27	rawData	2
ИТОГО	$\eta_1=25$	N1=144	ИТОГО	$\eta_2=27$	N2=145

Расширенные метрики

- Словарь программы: 52
- Длина программы: 289
- Объем программы: 1647.43 бит

Программа-парсер написана на HTML/JS.

Результаты работы программы представлен на рисунке 1.

Словарь операторов (η_1): 25

Общее число операторов (N_1): 144

Словарь операндов (η_2): 27

Общее число операндов (N_2): 145

Рисунок 1 – Результаты работы программы

Таблица операторов, получившаяся в результате работы программы, представлена на рисунке 2.

j	Оператор (f1j)	f1j
1	calculateMean()	2
2	for()	3
3	if()	9
4	calculateVariance()	2
5	findMinMax()	2
6	filterPositiveNumbers()	2
7	while()	1
8	result.push()	1
9	processDataset()	2
10	console.log()	8
11	main()	2
12	[]	9
13	=	30
14	<	5
15	.	6
16	+	9
17	===	4
18	/	3
19	-	2
20	*	2
21	>	4
22	function	6
23	let	20
24	return	8
25	else	2

Рисунок 2 – Таблица операторов

Таблица операндов, получившаяся в результате работы программы, представлена на рисунке 3.

i	Операнд (f2i)	f2i
1	string_literal	8
2	array_literal	6
3	data	18
4	totalSum	4
5	0	14
6	count	10
7	i	18
8	length	6
9	1	8
10	meanValue	2
11	sumSquaredDiff	4
12	diff	3
13	minVal	4
14	maxVal	4
15	result	2
16	index	5
17	current	3
18	numbers	2
19	filtered	5
20	dataLength	3
21	meanVal	3
22	varianceVal	2
23	minMax	3
24	score	4
25	50	1
26	20	1
27	rawData	2

Рисунок 3 – Таблица операндов

Расширенные метрики Холстеда представлены на рисунке 4.

Словарь программы (η): 52

Длина программы (N): 289

Объем программы (V): 1647.43 бит

Рисунок 4 – Расширенные метрики

Вывод: в ходе выполнения лабораторной работы было разработано консольное приложение на языке программирования TypeScript, реализующее вычисление основных статистических характеристик числового массива (среднее значение, дисперсия, поиск экстремумов и фильтрация). Программа демонстрирует использование основных конструкций языка: объявлений типов, циклов for и while, условных операторов if/else, массивов и пользовательских функций.

Для оценки сложности программного средства было разработано веб-приложение (парсер) с графическим интерфейсом на HTML/JS. С его помощью были рассчитаны базовые и производные метрики Холстеда. Проведенное тестирование и ручной пересчет подтвердили корректность работы лексического анализатора парсера при вычислении объема и сложности исходного кода.